QC Foundations [2/4]

Your First Quantum Speedup

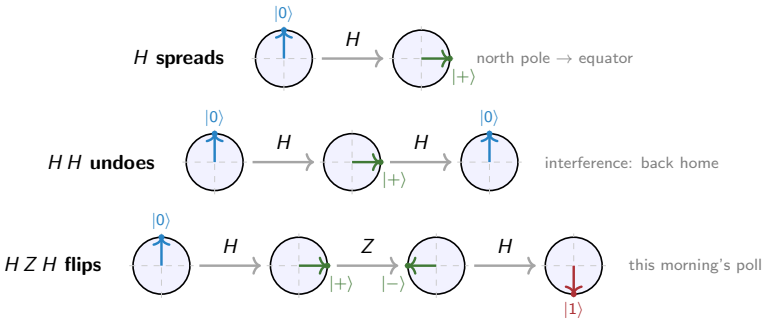
Quantum Computing — CEP

Kolin Paul

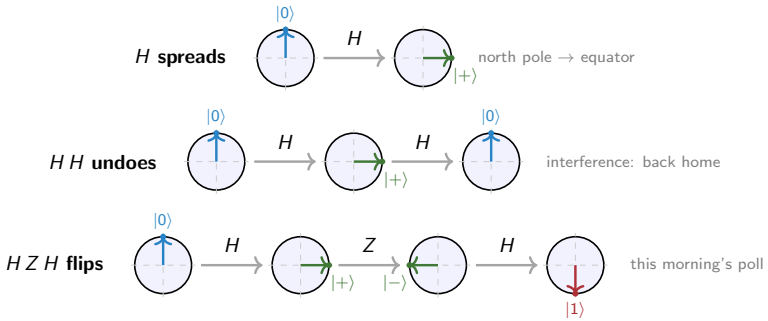
Department of Computer Science and Engineering
IIT Delhi

24 October 2026 — Afternoon

Three Moves from This Morning



Three Moves from This Morning



Hold on to the last row

Whether a hidden Z is there or not decides whether you read 0 or 1. That is today's whole algorithm.

The Coin Puzzle



A dealer hands you a coin, face up.
Is it fair or a trick?

The rules

- You may look at **one side at a time**
- Each look costs you money
- You must be **100% sure**

The Coin Puzzle

Fair coin



two *different* faces

Trick coin



the *same* face twice

A dealer hands you a coin, face up.
Is it fair or a trick?

The rules

- You may look at **one side at a time**
- Each look costs you money
- You must be **100% sure**

One look shows you **H**. That fits both coins.

You *must* turn it over: **two looks**.

You do not care what the faces are. You only care whether they are the same or different. That is a question about the coin as a whole.

The Black Box (an "Oracle")



Only four boxes are possible:

always 0	**always 1**	**copy**	**flip**
$0\to0$ $1\to0$	$0\to1$ $1\to1$	$0\to0$ $1\to1$	$0\to1$ $1\to0$

Constant — trick coin (same face twice)

Balanced — fair coin (one of each)

The task: decide constant or balanced. You may only feed inputs in and read outputs. You cannot open the box.

Poll — How Many Queries?

Question

A classical computer may query the box: send in x, read $f(x)$. How many queries does it need to be **certain** whether f is constant or balanced?

A) 1 **B)** 2 **C)** 3 **D)** It depends on the box

Poll — How Many Queries?

Question

A classical computer may query the box: send in x, read $f(x)$. How many queries does it need to be **certain** whether f is constant or balanced?

A) 1 **B)** 2 **C)** 3 **D)** It depends on the box

Ask $f(0)$. Suppose you get **0**.

always 0	✓ still possible
copy	✓ still possible
always 1	✗
flip	✗

Answer: B — 2 queries

One constant box and one balanced box are still in play. Only $f(1)$ can separate them.

The Quantum Promise

A quantum computer needs **one** query.

And for n-bit inputs it *still* needs only one.

What it is *not*

"Try both inputs at once and read both answers."

We'll see shortly why that fails.

What it *is*

Ask a question about the box *as a whole* (same or different?) and let interference produce the answer.

Plan: make the box quantum, build the circuit, run it, then see why it works.

Making the Box Quantum

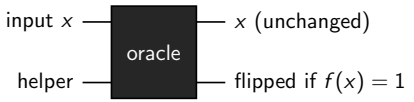
Problem: quantum gates are rotations, and every rotation can be undone.



✗

Output 0 — was the input 0 or 1? Lost. You cannot run this box backwards.

Fix: keep the input; write the answer on a second wire.



The rule, in words

Look at x. If $f(x) = 1$, flip the helper. Otherwise do nothing.

Run it twice and you are back where you started, so it is reversible. The formal rule $|x\rangle\,|y\rangle \mapsto |x\rangle\,|y \oplus f(x)\rangle$ comes in L3.

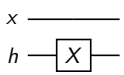
The Four Oracles as Circuits

always 0



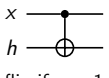
never flip

always 1



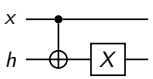
always flip

copy



flip if $x = 1$

flip



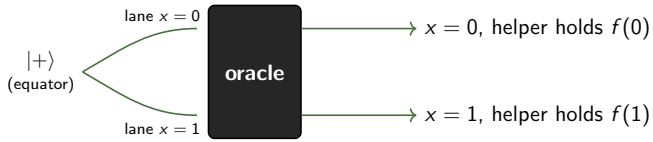
flip if $x = 0$

You already know every gate here

Constant oracles never touch the input wire. Balanced oracles need a CNOT: the helper's fate *depends on* x.

In real life the box could be millions of gates. We only care about its behaviour.

Quantum Parallelism



one pass through the box

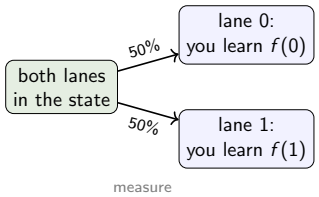
One query, both answers in the state

Send in an equator state and the box acts on both lanes in a single pass. The resulting two-qubit state "contains" $f(0)$ *and* $f(1)$.

You built this this morning! H then CNOT is the "copy" oracle fed a superposition, and the output is the Bell pair.

▸ Quirk: parallelism

The Catch — You Only Get One Shadow



Measuring casts a single shadow (this morning's analogy).

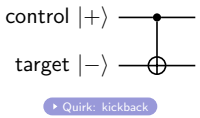
You get **one** lane at random, so you learn **one** value of f.

That is *worse* than a classical query: you didn't even choose which input!

Parallelism alone is useless

The information is in the state, but measurement cannot pull it all out. We need to make the two lanes **interfere** so that the *relationship* between $f(0)$ and $f(1)$ shows up in a single readout.

Poll — The Kickback Experiment

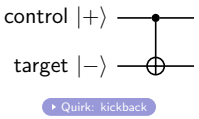


Question

Control on the equator at $|+\rangle$; target at $|-\rangle$. Apply CNOT. Which Bloch arrow moves?

A) target **B)** control **C)** both **D)** neither

Poll — The Kickback Experiment



Question

Control on the equator at $|+\rangle$; target at $|-\rangle$. Apply CNOT. Which Bloch arrow moves?

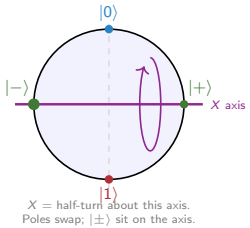
A) target **B)** control **C)** both **D)** neither



Answer: B — the *control* moves; the target doesn't

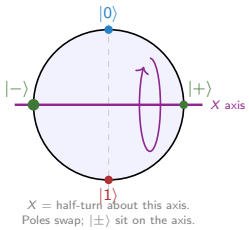
CNOT "flips the target", yet here the target stays put and the control is twisted from $|+\rangle$ to $|-\rangle$. This is **phase kickback**.

Why the Control Moves



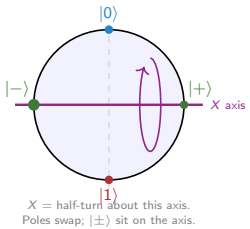
1. The control $|+\rangle$ is a 0-branch and a 1-branch side by side.

Why the Control Moves



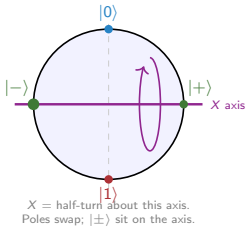
1. The control $|+\rangle$ is a 0-branch and a 1-branch side by side.
2. CNOT acts **only on the 1-branch**: it applies X to the target there.

Why the Control Moves



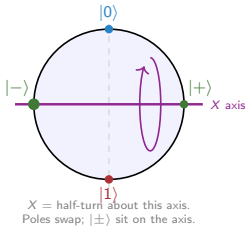
1. The control $|+\rangle$ is a 0-branch and a 1-branch side by side.
2. CNOT acts **only on the 1-branch**: it applies X to the target there.
3. The target is $|-\rangle$, **on the X axis**. The flip cannot move it; all it leaves behind is a **minus sign**: $X|-\rangle = -|-\rangle$

Why the Control Moves



1. The control $|+\rangle$ is a 0-branch and a 1-branch side by side.
2. CNOT acts **only on the 1-branch**: it applies X to the target there.
3. The target is $|-\rangle$, **on the X axis**. The flip cannot move it; all it leaves behind is a **minus sign**: $X|-\rangle = -|-\rangle$
4. That minus sign sits on the control's 1-branch. A minus on the 1-branch is exactly a Z: $|+\rangle \to |-\rangle$.

Why the Control Moves



1. The control $|+\rangle$ is a 0-branch and a 1-branch side by side.
2. CNOT acts **only on the 1-branch**: it applies X to the target there.
3. The target is $|-\rangle$, **on the X axis**. The flip cannot move it; all it leaves behind is a **minus sign**: $X|-\rangle = -|-\rangle$
4. That minus sign sits on the control's 1-branch. A minus on the 1-branch is exactly a Z: $|+\rangle \to |-\rangle$.

The helper is an anchor; the kick travels back up the wire

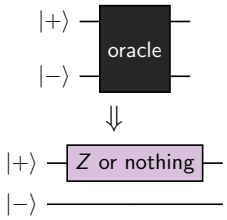
With the helper parked at $|-\rangle$, "flip the helper" turns into "twist the control".

The Kickback Rule for All Four Oracles

Helper parked at $|-\rangle$, input at $|+\rangle$:

Oracle	Input wire feels	Input ends at
always 0	nothing	$\|+\rangle$
always 1	a sign on *everything**	$\|+\rangle$
copy	a Z twist	$\|-\rangle$
flip	a Z twist*	$\|-\rangle$

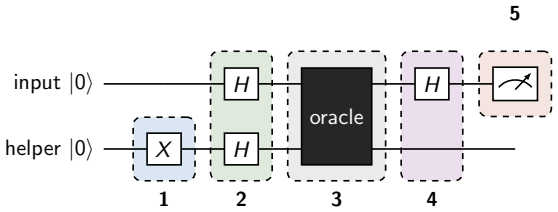
*A sign on the whole state is invisible to every measurement (global phase).



Constant ⇒ east ($|+\rangle$). Balanced ⇒ west ($|-\rangle$).

One pass through the box puts the answer into the input qubit's **longitude**. Now we just need to read longitude.

Deutsch's Circuit — The Blueprint

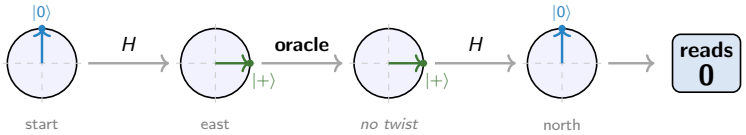


1. **Prepare:** helper to $|-\rangle$ (the anchor)
2. **Spread:** input to $|+\rangle$
3. **One query:** twist, or not (kickback)
4. **Decode:** H turns longitude into latitude
5. **Read** the input: **0 = constant, 1 = balanced**

The box is used exactly once.

Step by Step — A Constant Box

Watch the input qubit

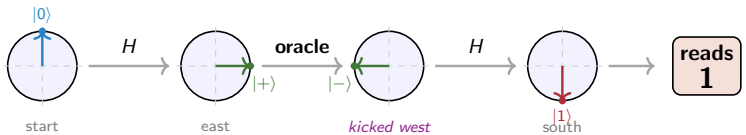


Every time, 100%

A constant box never touches the input wire (kickback gives at most an invisible overall sign). The input does H then H: this morning's "back home" move.

Step by Step — A Balanced Box

Watch the input qubit

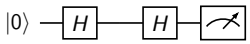


Every time, 100%

A balanced box kicks a Z twist onto the input. The final H turns "west" into "south".

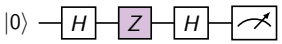
It's This Morning's Poll!

Constant box



$H\,H$: always reads **0**

Balanced box



$H\,Z\,H$: always reads **1**

Deutsch's algorithm in one sentence

The oracle secretly decides whether a Z sits between two Hadamards. The answer comes out with certainty after **one** query.

The helper wire is just the device that turns "flip if $f(x) = 1$" into "Z or no Z".

Live in Quirk — Four Oracles, One Query Each

Oracle	Type	Prediction	Run it
always 0	constant	reads 0	Quirk
always 1	constant	reads 0	Quirk
copy	balanced	reads 1	Quirk
flip	balanced	reads 1	Quirk

Poll

In "always 1" the helper *is* flipped. Why does the readout still say 0?

A) Quirk bug **B)** The flip only adds an overall sign **C)** The final H undoes it

Live in Quirk — Four Oracles, One Query Each

Oracle	Type	Prediction	Run it
always 0	constant	reads 0	Quirk
always 1	constant	reads 0	Quirk
copy	balanced	reads 1	Quirk
flip	balanced	reads 1	Quirk

Poll

In "always 1" the helper *is* flipped. Why does the readout still say 0?

A) Quirk bug **B)** The flip only adds an overall sign **C)** The final H undoes it

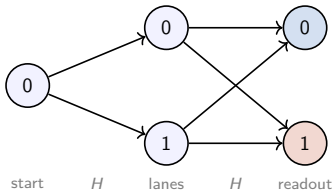
Answer B: a constant box treats both lanes the same, so any sign hits both lanes equally and nothing relative changes.

Break

10 minutes

Two Paths to Every Answer

The input qubit's journey



Each H is a fork in the road.
Two routes end at "0" and two end at "1".

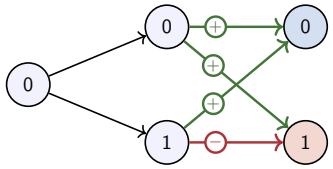
The rule (Feynman's arrows)

- Every route carries a little **arrow**
- Add the arrows of all routes to an ending
- Chance $\propto$ (length of the total)^2

→ + → = → long: **likely**

→ + ← = • nothing: **never happens**

Constant Box — Where the Arrows Go



oracle: no twist

Green route = arrow $\rightarrow$; red route = arrow $\leftarrow$. The second H reverses the route $1 \rightarrow 1$, as in this morning's $H|1\rangle = |-\rangle$.

Add up the arrows

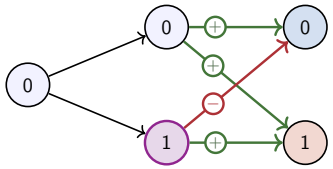
reach 0: → + → = →

reach 1: → + ← = •

Reads 0, always

The routes to "1" **cancel** (destructive).
The routes to "0" **reinforce** (constructive).

Balanced Box — The Oracle Flips One Lane



oracle: kick on lane 1

Every route through lane 1 has its arrow reversed by the kick.

Add up the arrows

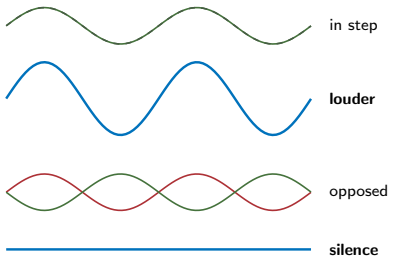
reach 0: → + ← = •

reach 1: → + → = →

Reads 1, always

Same routes, one arrow flipped:
now "0" cancels and "1" reinforces.

The Speedup Is Engineered Cancellation



Noise-cancelling headphones do the bottom row.

Not "try every answer faster"

A quantum algorithm arranges its routes so that the **wrong answers cancel** before you measure.

The three ingredients

1. **Spread**: H opens many routes
2. **Mark**: the oracle kicks signs onto some routes
3. **Interfere**: H recombines; signs decide what survives

Bernstein–Vazirani, Simon, Grover, Shor: same pattern.

Sandbox — Mystery Boxes (10 Minutes)

▸ Open the sandbox in Quirk (link in chat)

The template has Deutsch's circuit with an empty slot. Your toolbox has three custom gates: **Box A**, **Box B** and **Box C**.

Task 1

Drop each box into the slot. Run **once**. Constant or balanced?

Task 2

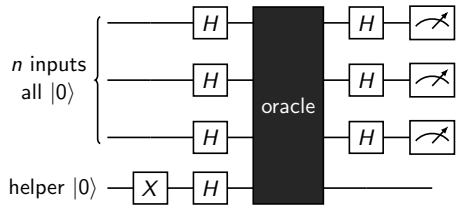
One of the balanced boxes contains *no CNOT*. Which one? How can it still be balanced?

Task 3 (stretch)

Remove the helper's X gate (so it starts at $|+\rangle$ instead of $|-\rangle$). What breaks, and why?

▸ Answers

From 1 Bit to n Bits — Deutsch–Jozsa



- Now f takes an n-bit input
- **Promise:** f is constant, or exactly half its outputs are 1
- Same recipe: spread, **one** query, interfere

Readout rule

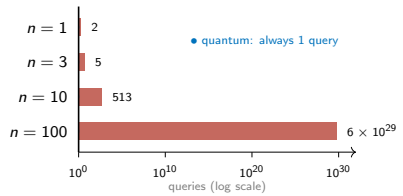
All inputs read **0** ⇒ constant
Anything else ⇒ balanced

Same picture as before, but with 2^n routes instead of 2. For a constant box every route to "all zeros" carries the same arrow, so they all add up. For a balanced box exactly half are flipped, and they cancel completely. The algebra is in L3.

The Query Race

n	**Classical (certain)**	**Quantum**
1	2	1
3	5	1
10	513	1
100	$2^{99}+1 \approx 6\times10^{29}$	1

Classically you may need to see *just over half* of the 2^n inputs: half could all agree by bad luck.



At a billion queries per second

$n = 100$ classically: about **20 trillion years**. Quantum: one query.

▸ Fine print

From Quirk to Qiskit

Deutsch in Qiskit

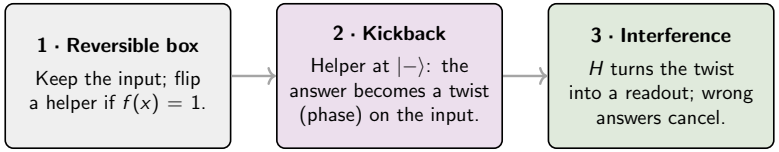
```
from qiskit import QuantumCircuit

qc = QuantumCircuit(2, 1)
qc.x(1)                    # helper to |1>
qc.h([0, 1])        # spread; helper to |->
qc.cx(0, 1)              # oracle:  "copy"
qc.h(0)                        # interfere
qc.measure(0, 0)             # read input
```

- Swap the oracle line to try the other three boxes
- Run 1000 shots: you should see **one** outcome, every time
- Qubit 0 is the input; qubit 1 is the helper

Qiskit lists bits right-to-left (little-endian), so the counts read `q1q0`. Quirk draws qubit 0 at the top. Same circuit.

Three Takeaways



One sentence

Quantum speedup is not about trying every option at once. It is about making the wrong options **cancel** before you look.

Reading

Mermin (start here)

- §2.1: general features of quantum algorithms
- §2.2: Deutsch's problem (today, in Mermin's notation)

Nielsen & Chuang (deeper)

§1.4.2 quantum parallelism; §1.4.3 Deutsch's algorithm; §1.4.4 Deutsch–Jozsa

Mermin and N&C use the algebra we postponed. Skim them now for the pictures, and read them properly after 31 October.

Quirk

algassert.com/quirk

All of today's circuits are in the chat.